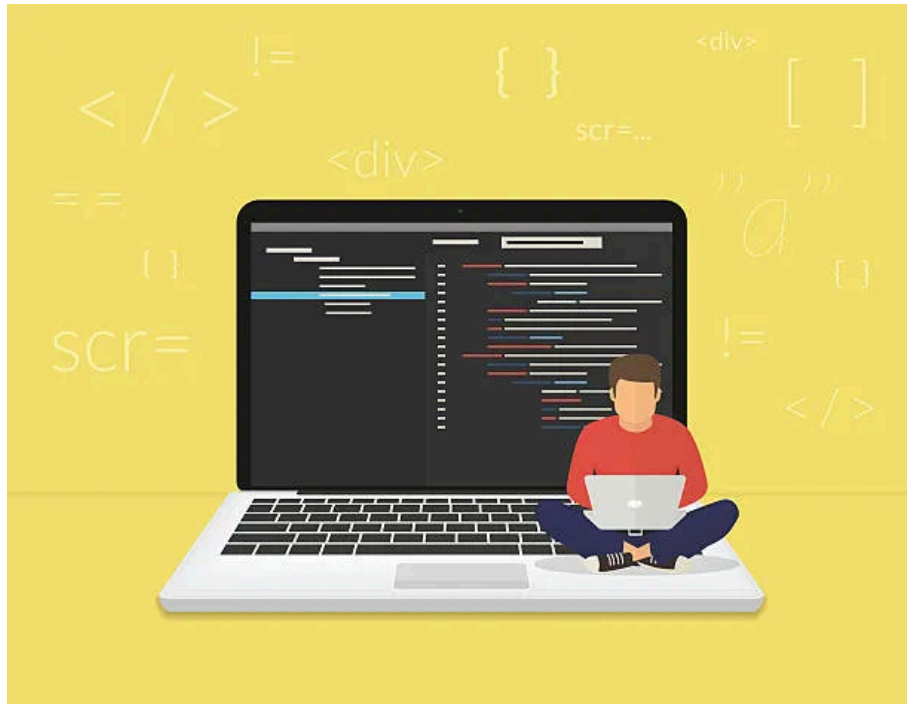


Deliverable 3: Documentation



Autors

Pol Ortega Fernàndez: 49254483Y

Andreu Puig-gròs Munté: 48255353L

Marc Pujol Navajo: 49259109D

Escola Politècnica Superior

Work presented to: Roberto Garcia and David Sarrat

Subject: Web project

Course: 2025-2026

Date of delivery: 7 of June, 2026

Contents

Introduction.....	1
1. Github public address.....	2
1.1. Project Source Code & Open Source Repository.....	2
1.2. Accessing the Repository.....	2
1.3. What can be found in the repository.....	2
1.4. Users.....	2
2. Notes about all the design decisions.....	3
2.1. Web 3.0 Application Development (Markup in the HTML).....	3
2.2. Matchmaking primitive functionality.....	5
2.3. Other design decisions and features added.....	7

Introduction

The following report provides a comprehensive yet simple technical breakdown of the development process for the Django-based web application presented in this subject. It's designed to facilitate the access and data management of a competitive Counter-Strike 1.6 related website that is exclusively focused on 1v1 game matches inside the web browser along with ranking and customisation features.

This project was developed as a direct evolution of the previous deliverables, incorporating the constructive feedback and technical requirements to ensure the robust final product. While the second deliverable focused on the implementation of our review system, database fixtures, as well as initial game server integration, this third deliverable represents the evolution of the platform into a Web 3.0 application. To achieve this, some basic semantic web mechanisms have been incorporated into the project. We've specifically implemented semantic markup based on RDFa within the reviews' list and details pages of our review entities by using the standardised types and properties from [Schema.org](https://schema.org).

Furthermore, this structured data has been successfully tested and verified to be properly recognised by the Google Structured Data Evaluation Tool (with a specific exception that we'll talk about inside its corresponding section in the report), adhering strictly to the guidelines provided in the course's Web 3.0 documentation. Also, the website has been fixed and modified with some extra features around the whole site with the objective to improve the QoL of the users who play on the web page, making their experience more attractive and comfortable with the website. In order to ensure the high standards of software portability and scalability, the app still remains containerised using Docker and orchestrated via Docker Compose, building upon the deployment foundations previously established. This last documentation provides a thorough overview of our final proposed solution. It includes the updated data model documentation, the newly integrated semantic markup design decisions, the new complete matchmaking functionality, as well as other design decisions' explanations and the validation of the Web 3.0 features of our application.

1. Github public address

1.1. Project Source Code & Open Source Repository

The development of the Strike 1v1 project is publicly accessible on the GitHub platform. By doing it this way, we can allow everyone to see the main web page's source code as well as, once finished, the implementation of other features such as matchmaking and the ability to play through the website. Also, by hosting our source code on GitHub, we can also let anyone interested review our multiple open issues, pull requests, and all the program logic. They will also be able to track the whole evolution of the platform from its alpha stages (preliminary) to the production environment (advanced).

1.2. Accessing the Repository

The GitHub public address for the project's source code is the following one:

- **GitHub URL:** <https://github.com/MarcPujolUNI/Strike1v1>

Also, we provide the link to the Strike 1v1 server, where we are developing the server to play:

- **GitHub URL:** <https://github.com/Apm-96/1v1Strike-server>

1.3. What can be found in the repository

The core engine of our website has been implemented using the Django framework, obviously with its latest version, 6.x. All the frontend and backend elements in our project are based on this framework. The frontend assets will let you see the Tailwind CSS configuration, which allowed the development of the website's main design features. The design is based on a retro style with some added modern elements that cause the website to have the typical look that these kinds of platforms normally have.

Furthermore, all the database migrations are also available. Those are the main database tables that the platform will use to ensure that all the features the project wants to achieve are indeed completed.

1.4. Users

Since this release will be made available to people so they can play and test the website, the users will be changing throughout the project's life cycle. We'll begin having the same users as before, but we'll then change the passwords of our own users (to make it safer) as well as the admin's one (our only superuser for now). We'll be deleting the matches written (both test ones and hardcoded ones) along the hardcoded reviews to leave space for the real ones from real users, and we'll also reset all the scores and attributes from the current users, keeping only both the countries and the maps intact.

2. Notes about all the design decisions.

In the following section, we will visit each one of the main points related to the new features that we've added, and we will also see why all of those design decisions were made in this last third deliverable.

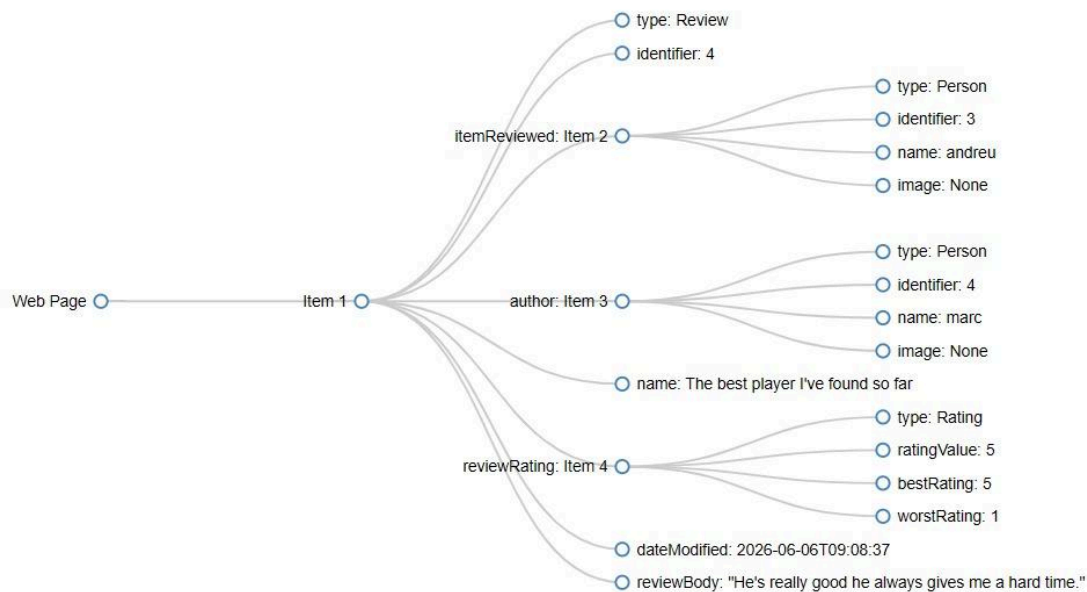
2.1. Web 3.0 Application Development (Markup in the HTML)

We'll begin with the main feature asked for this deliverable: the web 3.0 markup feature. For this, we chose to annotate the review entity's html, as noted previously. We first chose the reviews details html to annotate, which contains the important information about each review and is in charge of displaying it all in the corresponding specific review's page. This template contains 2 main sections scripts aside, the delete one, that was left untouched, and the main one, that contained all the desired information.

This contains a handful of divisions inside, but since the review's information is scattered along more than one of them, we defined the property in the main division with `typeOf = Review`, as well as noting that we're using vocabulary from Schema.org. In the next division, we have the reviewee's data, and we used the property `itemReviewed` to note we're reviewing a Person. We could have also defined it a bit differently, with the main entity being Person (the reviewer) adding another layer, and then having Review noted with `rel`, which is the structure we're using for the next html template. Since this is only one review, we actually want to focus on it. Sadly, both the schema.org and Google's validators aren't ready for this kind of review system, it seems they don't like People's Reviews (neither having Person as `itemReviewed`, nor as a parent node): Google's marks it as a severe mistake, whereas schema.org is more forgiving, so it notes it as a minor mistake. Still this is the meaning we have in our website, so we left it like that. Unlike before, we create a new span label and define the property here, and begin using meta labels since we don't have the actual values listed here, we're adding this information but we don't want it to be displayed. We'll be doing this along the content parameter several times during this deliverable, in order to control the order the attributes get displayed in the graf and because of the lack of information otherwise. Person needs an identifier (WebUser's id) and a name (since this one is being displayed (using `review.reviewee.name`), we can use the span label (there was no label here), and omit the content parameter. We added an if structure to show information about the user's image, writing None if it doesn't have any, and the path saved in the WebUser's instance if it has one defined.

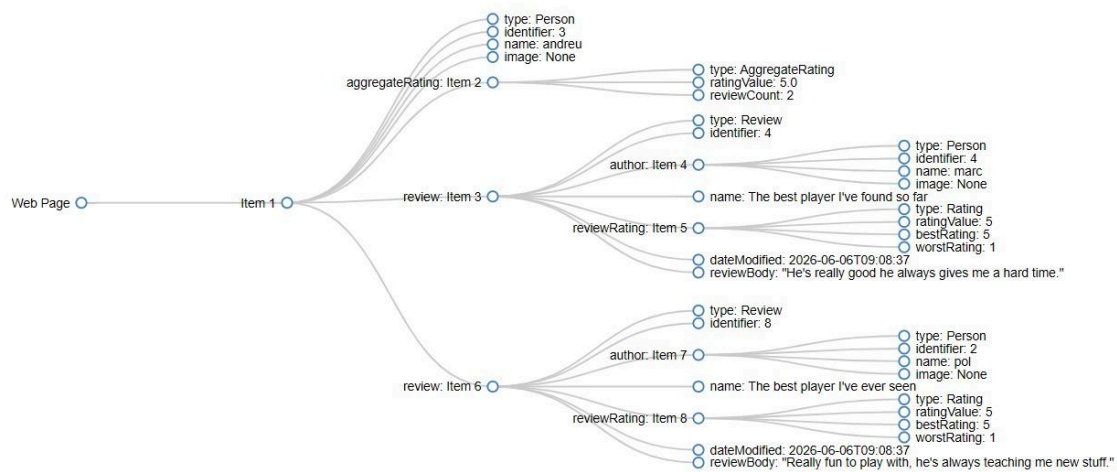
We'll be using this structure several times in the future, with every person, but we use meta here as we didn't have this information before, and the user shouldn't see that. We also defined the review's id above that (property), using meta as well. This section is in the top blue area of the review's display in the website. The next part commented, is the one where both related users are displayed again along the reviewer's image. Since we have more information here, we'll define the author property, referring to who wrote the specific review, instead of doing it before. We only have to annotate everything once, it doesn't matter if the values show up more. We can define the image property by taking advantage of the structure that's already created to display it here. We added code for the else case with a meta label (again) defining the property from scratch, unlike before (we wrote it inside an existing division).

We don't have the review's title close though, but as we wanted this to go before the rating, which is close enough, we just added another meta line with the content being `review.title`. We can now define the rating parameter (`reviewRating`) which is another entity of its own. We made things easy and used the meta label to define the actual rating, but also the worst and best ones (possibly, the range, that goes from 1 to 5, like in the example provided in class) (the rating is shown below, but the structure is a bit convoluted). We then have the `dateModified` attribute, from Review's `lastModified` attribute, which was added in this deliverable, added with a meta label in between two sections, before the code that actually uses it (in the middle of the screen), and a bit below, we have the `reviewBody`: we just added the property label in the line that was already showing it. We obtained the following structure, for one specific example, after executing the code, fetching the specific html and copying in the RDFa page:



We later added annotations for the reviews list template, in order to have a more general vision of the reviews from a specific user. We only have one main division here, excluding the script part, so we will define the vocabulary parameter here, but with `Person` this time, since we'll make this revolve around the person being reviewed (`reviewee`). We use a meta line to define its identifier immediately and we'll take advantage of the top blue display print to define the reviewee's name, using `span` labels and the corresponding property, `name`. We don't have the image here, but we'll still use the same if structure from before to check if the user has defined itself an image, but with meta lines with content and the property `image` this time. We'll then handle the `aggregateRating` property, something added in this last deliverable as well, inside the next division, reusing an existing line. We can use `span` for `ratingValue` as it's being displayed in the html, right below the `Community Feedback` title on the screen, handling the `reviewCount` one similarly, which actually refers to the `total_reviews_count` property we calculated before (`ratingValue` refers to `avg_rating`) in the html. If we have no reviews we'll define default values for those attributes, both zero, with meta lines. After some html code related to forms, for when the user accessing hasn't left a review yet (so it can leave one), we'll find the code for its review, if it has written one. We'll use `rel` inside the division line that contains all the information needed. After `typeOf`, obviously, we'll define with a meta lin, the identifier attribute (`review id`) as it's never shown code-wise.

We'll then define the author property (reviewer (WebUser), the one who wrote the review), and since the image is displayed here, we'll have property in the image line and a meta line with content None in the else case. The identifier and name will be defined with meta labels and content to match the order defined previously, and to mess as little as possible with the html code structure. We'll handle the reviewRating property the same way as before, we'll take advantage of an existing line for the review content (body) and put the meta line for dateModified right between those 2, between 2 divisions, like before. Then, we have code for the other reviews, not written by the user, and they'll all be treated the same way as they're inside the same loop (after the filter html section). This works very similarly to the previous reviews: rel with typeOf Review, instantly adding the meta line with identifier and content id, then the author again (having the name in an existing line (span) and identifier in a new one (meta)), but with both image cases with meta lines (those images aren't being displayed). After the review title, we have the rating property (same as before), review body at the end and dateModified 3 lines before. We can see the result for a specific user with the same process as before: the image is right below.



2.2. Matchmaking primitive functionality.

Since the derivable 2 there have been a lot of changes, we'll explain them all. First of all, we completely wiped out the project's last implementation of the matchmaking feature (which had the database as temporal storage, a bad and unoptimized decision). The alternative was to implement Redis, to manage the queues of the process. We have also implemented a locking mechanism to take care of running conditions when 2 players are trying to match at the same time, guaranteeing that only one server assigns only one pair of players (also securing multithreading. With this implemented we have to find the correct player to compete. To avoid unlimited waiting time when having players on the queue, the search uses an expansion algorithm where the elo margin starts low (+-100 ELO) and expands every 30 seconds dynamically depending on the "attempts" of the player stored inside Redis. There's also a timeout and cancellation logic if the player spends too much time on queue to avoid ghost players. One of our current implementation's bottlenecks was the communication between the container where the match is stored, the custom API for container communication and then the site.

Firstly, to communicate the container and the API inside the same PC/server we used host.docker.internal, which functionally is a little tricky, but represents the internal loopback of the main machine, avoiding opening a new port. Also we updated the API to handle "RequestExceptions" avoiding to break the site if the game server breaks. With all of this implemented, we encountered a rather difficult error to debug, an internal error when sending the JSON with all the information of the match inside of the container. At first look it doesn't seem to be a big problem, but for some reason any prints or debugging techniques didn't work, so we had to investigate the root of the shipping process of this message. Thanks to a webhook.site we sent the JSON on their API and we found out that the JSON itself was in the wrong format, it had american-like date, and an extra ":" was present on the starting time variable (accidentally). The log also had problems with special characters, which messed up the format. After that slip-up, we ensured that the Match, MatchStats and ELO happens to be an atomic transaction inside the database. We also finished implementing the log cleaner: match logs are the biggest information that we have and we don't have unlimited space inside our humble server, so it erases those logs if they are 30 days old or more. In order to know when to launch this cleaner, we used the service cron (linux), which wakes up every certain time and executes the method in parallel.

As said beforehand, we fixed the function in charge of creating Matches and MatchStats using the django data, extracting everything, querying to obtain the counter user's objects (winner and loser), but the map as well, fixing the data and duration formats, and finally creating the match related items, which will trigger all the update logic from the database. We also finished the scoring function, used to determine the amount of points won and lost for each player. We obtained those parameters through trial and error, testing some cases and seeing the output values to see if they were fitting or not, and we started building everything on top of the exponential function idea and score difference, so that players with higher elo gain less and the higher the difference, the more impactful wins are also for the user with less points (and losses for the higher rated). Finally, the last thing implemented related to matchmaking was to hide the modification of the player's name. As the repository we run the CS from is setted up, when we enter the container it shows a space box to enter your name inside the game. This feature would break the functionality of our data management if left unchecked, as the user would be able to put anything and we wouldn't find the user object later. So, we have to avoid this. The solution we reached might not be the best but it was the best option we had with the limited time remaining. It consists of sending the registered player name inside the url, following the format below, for example: "/username=patata". This was a very bad solution at first because it is very easy to change the value and play for someone else, so we thought to encrypt the name in this equality, this way it would stay like this: "/username=bfjsfaje4fjn". The user can still modify his username on his url but it doesn't know which letters are which. We'll now sum up the workflow of the matchmaking: it's an ecosystem where a user enters the website, gets queued, Redis finds an opponent at their skill level, a game server is automatically spun up, the user connects with their name already configured, and when the match ends, the result is stored in the DB along its log. If you are interested on the server-side of the project, it's on the following repository: <https://github.com/Apm-96/1v1Strike-server>

2.3. Other design decisions and features added.

Now we are going to talk about other minor features and design decisions added during this deliverable. We will see some of the fixes implemented and a pair of new feats added to the website.

In this case, we have introduced some fixes related to pagination numbers. The main reason is that with the old number that was defined, users had to do a lot of scrolling to get to the bottom of the page. For that reason, we reduced the number of the pages to avoid this scroll. Also, to improve the QoL for the users, now in the leaderboard, the top 3 users of each ranking have the typical 3 colours of gold, silver and bronze. Making it easier to know who the best players are on the website.

Another important fix that has been implemented is that on the review detail page, if the users wrote a very long word, it could easily get outside the defined box to display the text. Additionally, something similar happened in the "summarised" version of the review list page. Now it also has been fixed, and a truncated section has been added so that if the message is too large, now it shows ... to summarise. In the same pages, at the review list page, a button has been added to return back to the user's search page. This has been introduced to help the users to return easily to this page without having to press again the button at the header of the website.

At the user's search page and country search forms, where the users could type the name of the user they wanted to review or the country they wanted to be part of or filter by. In the previous deliverable, the API we defined internally for the website was using the function *icontains* to search for these terms. The problem with it is that it normally was showing more options than the expected ones. The main reason is that it is easier for a word to contain a specific sequence of letters rather than start with them. So, to solve this and make the API more specific, now it searches the terms by *istartswith*. Doing this, the API is more specific and shows better results for what the users are trying to search.

On the user profile page, now a button has been added to delete the user profile image. It appears when the user has uploaded an image for their profile, and if they don't like it, they can delete it easily. Moreover, the username in the profile page and dashboard now is displayed as how the user wrote it. Other fixes introduced in the profile page are related to the display of the favourite map of the user. In the previous deliverable, the main problem was that it was not being correctly captured from the website database, and this field was always displayed with the default option. Now, this issue has been fixed. Also, in the display form that appears to select the user's favourite map, an option with the placeholder text we defined appeared. This has been solved to just show the maps that are playable.

Throughout the website, there are a lot of buttons to facilitate the navigation through the pages. One of the issues with it is that some buttons had a different style, others had a different hovering, and there were other related problems. Now, all the buttons have been changed to have the same style and hovering, making it easier for the user to identify these buttons through the website navigation.

One of the features that have been introduced to this deliverable is that now on the register page, the default country is already preselected. This way, if the users don't want to give us their origin country, they can leave this option with the default country. To change it, they delete it and add their country.

Another of the features is that on the review list page, now a button has been added to sort the reviews by date they were created or number of stars, ascending or descending, respectively. To make this sort button work properly, the models of the website have been modified to introduce the date and time a review was created. Moreover, to improve the QoL of the users, the time a review was created or modified is now displayed in each one of the reviews on the list page and also inside the review detail. Additionally, it has introduced the total number of reviews a specific user has and the average rating, which shows the rating a user has combined from all the stars that other users give to him. Making it easier for the users to know how the player behaved to the other users.

Another feature related to the reviews is that now you can only review players of the website that you have played versus. It has been added as a check to verify if you're leaving a review for someone you've actually played at least once. If you didn't, the button to leave a review will not appear. And if you can bypass it and send a POST with the review data, this is also checked internally to see that the user has played once with that user. If you didn't, you will be redirected automatically to the review list page, and the review you tried to send will not be processed and stored in the website database.

More features. In this deliverable, now superusers and staff members from the website are automatically redirected to the `/admin` page. Why? The main reason is that they are not actually current players from the web page, so they don't have any reason to access the different URLs that normal users can. So, when an admin user tries to access a URL that was originally designed for the normal users, they are automatically redirected to the `/admin` page again. Basically, now we handle the admins as a special user case. To do it, it has been clearly separated superusers and staff members from web players. To make it possible, middleware has been introduced to guarantee that admin users can only access `/admin` pages and web users only the website pages.

The final feature implemented is that a user section has been added with a specific dashboard to allow users to see other users' performance data and a button to redirect to the review list page of that user. Basically, in the second deliverable, we introduced the reviews section in the header. Now, it has been slightly modified to the users section. There you can look for any user on the website, and when you press the button to find the profile, you will be sent to a page where you will see user performance data. Previously, in other deliverables, users were not able to see other players' performance data. Now this issue has been fixed to allow all the users to see other players' data. Additionally, and already introduced, on this page where you see the user performance data, you will have a button to the review list page of that same user. We have decided to merge these two features to avoid having two different search pages in the header with a very similar functionality. Furthermore, in the profile page and matches history page, a button has been added to visit directly the user stats page. Following the same approach as the button to visit directly the review list page of the user.